

O loop mínimo de pergunta e resposta

Como o ciclo funciona: o humano pergunta, o modelo responde, o transcript acumula as duas falas, e o orquestrador volta ao início. Não há tools, store externo nem orçamento. O sistema é estado + uma chamada ao modelo + um loop.

Um chatbot de um único request não tem loop: pede e acaba. Aqui o loop existe para que cada nova pergunta veja tudo o que já foi dito. O modelo não lembra sozinho; ele só vê o que o orquestrador envia.

1. O que o loop é

O modelo é uma função sem memória: recebe uma lista de mensagens e devolve texto. Quem lembra é o host. O loop é o mecanismo que, a cada pergunta, acrescenta essa pergunta ao histórico, chama o modelo com o histórico inteiro, acrescenta a resposta, e espera a próxima pergunta.

Sem o loop, cada chamada é isolada. Com o loop, a conversa é um único estado que cresce. A diferença entre “chatbot de um tiro” e “conversa” não está no modelo; está neste ciclo.

- O modelo não age no mundo: só emite texto.
- O orquestrador não interpreta o conteúdo: só encadeia estado e chamada.
- A memória operacional é o transcript, não uma variável à parte.

2. Os quatro blocos

O sistema cabe em quatro peças. Cada uma tem um lado de dentro e um lado de fora. Se um bloco fizer o trabalho de outro, o ciclo deixa de ser legível.

Bloco	Responsabilidade	Fora do bloco
Transcript	Guarda as mensagens na ordem em que aconteceram.	Não chama o modelo. Não decide quando parar.
Adaptador do modelo	Traduz a lista de mensagens no formato da API e devolve o texto.	Não conhece o loop. Não grava estado.
Orquestrador	Dono do ciclo: lê, grava, chama, grava, mostra, repete.	Não executa ações no mundo. Não parseia a resposta para “fazer” algo.
I/O	Entrega texto do humano e devolve texto do assistente.	Não conhece mensagens, roles nem o vendor do modelo.

Dependências

```
I/O —> Orquestrador —> Transcript
      |               |
      |               |> Adaptador —> API do modelo
```

As setas apontam para dentro. O adaptador não importa o transcript. O transcript não importa o I/O. O orquestrador é o único que vê os três. Por isso ele é o loop: é o único lugar onde “depois disto, aquilo” está escrito.

3. Estado

O estado é um array de mensagens. Não há run, orçamento, pause nem índice de turno. Se algo precisa ser lembrado no próximo ciclo, está neste array. Se não está, o modelo não vê.

Campo	Papel
role	Quem falou. Três valores: system, user, assistant.
content	O texto dessa fala. Neste recorte, sempre uma string.

Os três roles

role	Origem	Função no ciclo
system	O orquestrador, uma vez, no início.	Instrução permanente. O modelo trata como contexto, não como fala do usuário.
user	O humano, via I/O, a cada turno.	A pergunta atual e todas as anteriores.
assistant	O modelo, via adaptador, a cada turno.	A resposta que o próximo turno precisa “lembrar”.

O system existe antes do loop começar. Cada volta bem-sucedida acrescenta exatamente duas mensagens: uma user e, depois da chamada, uma assistant. O array só cresce; mensagens antigas não se editam.

Há uma única memória. Qualquer cópia paralela (“última resposta”, “resumo atual”) vira uma segunda fonte de verdade e o ciclo passa a mentir quando as duas divergem.

4. O ciclo

O orquestrador é uma máquina de dois estados visíveis: esperando o humano e chamando o modelo. O transcript muda só nestas transições.

```
iniciar
  transcript ← [system]

enquanto verdadeiro
  pergunta ← I/O.ler()
  se a pergunta encerra a sessão → sair do loop

  transcript.append({ role: user, content: pergunta })
  resposta ← adaptador(transcript)
  transcript.append({ role: assistant, content: resposta })
  I/O.escrever(resposta)

encerrar
```

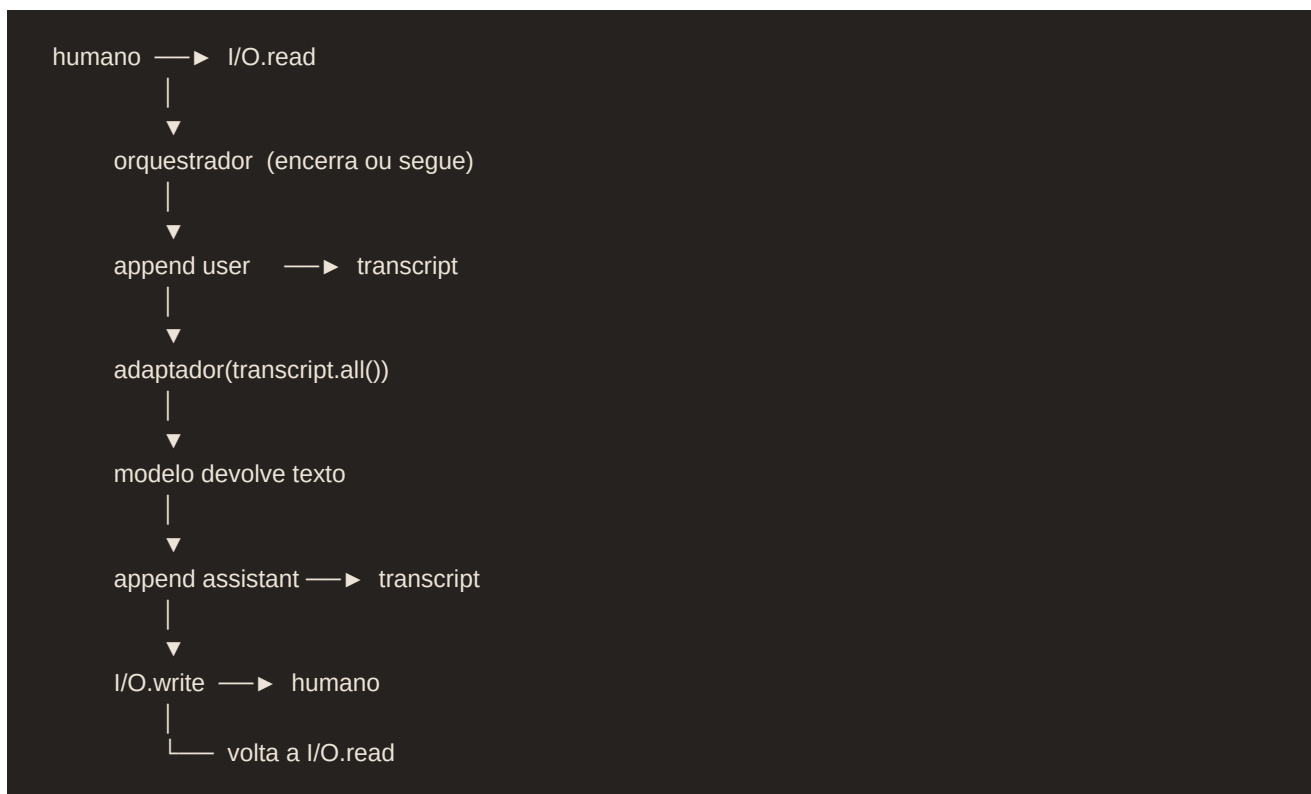
Invariantes

1. O modelo só é chamado depois do user estar no transcript. Sem isso, a pergunta atual não existe para ele.
2. A resposta só é mostrada depois do assistant estar no transcript. Sem isso, o próximo turno não vê o que foi dito.
3. A chamada envia o transcript inteiro, na ordem. O modelo não recebe um recorte “inteligente” neste recorte.

Uma pergunta produz uma chamada e um par user/assistant. Não há paralelismo: o próximo user

só entra quando o assistant do turno atual já está gravado, ou quando o turno foi abortado sem commit.

5. Anatomia de um turno



O turno é a unidade de consistência. Ou o par user/assistant entra inteiro, ou não entra. O adaptador devolve texto ou falha: não devolve uma mensagem pela metade para o transcript.

O I/O pode ser um terminal, um socket ou um handler HTTP. Para o orquestrador isso é irrelevante: ele só vê duas funções, ler e escrever string.

6. Contratos internos

Estes são os tipos que os blocos trocam. Não descrevem um vendor. Não há ToolCall: a saída do modelo, neste recorte, é só texto.

Contrato	Forma
Message	role: system user assistant content: string
Transcript	messages: Message[] append(message) all() → a lista que o adaptador envia
ModelAdapter	entrada: lista de mensagens saída: string falha de transporte: erro, não uma Message
Orchestrator	start() coloca o system turn(userText) → assistantText o loop externo chama turn até parar
I/O	read() → string write(string)

turn é o ciclo de um turno, não o loop da sessão. O loop da sessão é “enquanto I/O.ler() não encerrar, chamar turn”. Separar os dois evita que o adaptador ou o transcript saibam o que é “sair”.

7. A mensagem system

O system é a primeira mensagem do transcript e a única que não vem de um turno. Ela é estado, não configuração escondida no adaptador: viaja junto com user e assistant em toda chamada.

Exemplo do slot, não de um produto: “Responda em português, de forma direta. Se não souber, diga que não sabe.”

Arquiteturalmente o system é só um role com privilégio semântico no modelo. O orquestrador não o reescreve a cada turno. O adaptador não o inventa. Se a voz ou as restrições mudam, muda a mensagem no transcript — a mesma fonte de verdade.

O system não é barreira. Ele sugere comportamento. Quem impede uma chamada ou uma saída é o orquestrador, não o texto da primeira mensagem.

8. Parada

Neste recorte o loop não decide sozinho que “já acabou”. Quem encerra é o humano, ou uma falha que impede o turno de fechar. Não há `max_steps`: cada volta espera um input explícito.

Condição	Onde	Efeito no estado
Sinal de encerrar (sair, EOF, hangup)	I/O → orquestrador	O loop para. O transcript fica como está. Não há chamada ao modelo.
Falha do adaptador	Adaptador → orquestrador	Não nasce assistant. O turno não se completa.

Falha e o user órfão

Se o orquestrador já gravou o user e o adaptador falha, o transcript fica com uma pergunta sem resposta. Isso quebra o ritmo do array (user, assistant, user, assistant...). Há duas formas consistentes:

- Abortar o turno: remover o user recém-gravado. O array volta ao estado anterior à pergunta.
- Manter o user e não inventar assistant. A próxima tentativa ou o próximo envio precisa lidar com um user sem par.

O que o ciclo não faz: gravar um assistant com a mensagem de erro como se o modelo tivesse falado. Erro de transporte não é fala. Se o humano precisa ver o erro, isso sai pelo I/O, fora do transcript, ou como um canal à parte — não como role assistant.

9. Fronteira deste recorte

O recorte termina onde o modelo deixa de ser “função de texto”. Tudo abaixo existe em arquiteturas maiores; aqui não faz parte do ciclo.

Fora	Por quê
Tools	A saída deixaria de ser só texto. O loop teria um ramo “executar” antes de voltar ao modelo.
Store externo	Haveria uma segunda memória além do transcript. O compilador de contexto teria de juntar as duas.
Orçamento (passos, tokens, custo)	O orquestrador passaria a encerrar sem input humano. Aqui quem limita é o próprio turno.
Pause / resume	O run teria identidade própria, serializável no meio de um ciclo. Aqui a sessão é o próprio processo + o array.

A fronteira não é uma lista de tarefas. É o que este ciclo, por construção, não contém: um único estado, um único tipo de saída, um único dono do “e agora?”.

10. Por que a memória funciona

O modelo não “lembra” a conversa. Na segunda pergunta ele só acerta o contexto se o orquestrador reenviar o array completo. O exemplo abaixo é o ciclo visto pelo estado, não um roteiro de uso.

Momento	O que o transcript contém	O que o modelo recebe
Início	[system]	Ainda não é chamado.
Depois da 1ª pergunta	[system, user1]	system + user1
Depois da 1ª resposta	[system, user1, assistant1]	— (turno fechado)
Na 2ª pergunta	[system, user1, assistant1, user2]	as quatro mensagens

Se na segunda pergunta o modelo ignora o que foi dito na primeira, o defeito está no host: o adaptador não recebeu o array inteiro, ou o assistant anterior nunca foi gravado. O modelo não tem outro canal de memória neste desenho.